Saving Game Data in Unity

유니티에서 게임 데이터 저장하기 - PlayerPrefs부터 Newtonsoft JSON까지

You will learn how to store and retrieve data for your Unity game.

Gemini Notebook

Data Save and Load Overview

PlayerPrefs

유니티에서 기본적으로 제공하는
데이터 관리 클래스.

JSON

JavaScript Object Notation.
경량의 데이터 교환 형식.

PlayFab

클라우드 기반
백엔드 서비스.

Database

서버 연동을 통한
데이터베이스
(PHP & MySQL 등).

Gemini Notebook

Basic Data Management: PlayerPrefs

기본 특징

- ✓ 유니티에서 제공하는 클래스 사용.
- ✓ int, float, string, bool 타입의 변수를 Save/Load 하는 기능을 제공.

한계점 및 대안

- ⚠ 한계점 1: 가독성이 떨어짐.
- ⚠ 한계점 2: 저장하는 데이터 타입에 제한적임.
- ⚠ 한계점 3: 보안상 문제가 있어 해킹을 당할 수도 있음.

대안 (PlayerPrefsExtra):
Vectors, Bools, Colors, Lists 등 더 복잡한 데이턴 데이터 타입을 저장할 수 있게 해주며 내부적으로 PlayerPrefs를 사용함.

Gemini Notebook

PlayerPrefs Setup: UI & Variables

Name

Age

Height

Save

Load

```
1  using UnityEngine.UI;
2
3  public class DataManagerWithPlayerPrefs : MonoBehaviour {
4      public InputField inputName;
5      public InputField inputCount;
6      public InputField inputScore;
7
8      void Start() {}
9      void Update() {}
10 }
```

Gemini Notebook

PlayerPrefs: Execution & Registry Path

```
1  // Write data into Registry
2  public void Save() {
3      PlayerPrefs.SetString("Name", inputName.text);
4      PlayerPrefs.SetInt("Count", int.Parse(inputCount.text));
5      PlayerPrefs.SetInt("Score", int.Parse(inputScore.text));
6  }
7  // Read data from Registry
8  public void Load() {
9      if(PlayerPrefs.HasKey("Name")) {
10         string getName = PlayerPrefs.GetString("Name");
11         int getCount = PlayerPrefs.GetInt("Count");
12         int getScore = PlayerPrefs.GetInt("Score");
13         print(getName + " " + getCount + " " + getScore);
14     }
15 }
```

레지스트리 편집기 경로

Gemini Notebook

JSON Fundamentals (JavaScript Object Notation)

- 경량의 데이터 교환 형식 (a lightweight format for storing and transporting data).
  서버에서 웹 페이지로 데이터를 보낼 때 자주 사용됨.
- 자바스크립트 객체와 같이 Key/Value가 존재하며, 항상 쌍따옴표(“”)로 표기.
- 객체, 배열 등 원하는 만큼 중첩(Nested) 표기 가능하여 읽고 수정하기 편리.

JSON vs XML

JSON	XML
개방형 표준 텍스트 기반으로 이해하기 쉬움. 직렬화(Serialize)/비직렬화(Deserialize) 함수를 통해 데이터 변환이 매우 편리. 가독성 우수. 네트워크 게임 데이터 교환에 최적화.	가독성이 떨어짐. 데이터를 파싱(Parsing)하는 과정이 까다로움.

Gemini Notebook

JSON Structure Examples

test.json

```json
{
  "name" : "YongHwan",
  "playingdate" : "2023-11-10",
  "id" : "hwany1458",
  "snake" : [10, 20],
  "rsp" : [1, 2, 3],
  "owl" : [7.8, 10, 5, 100]
}
```

score.json

```json
{
  "name" : "YongHwan Lee",
  "affiliation" : "Wonkwang University",
  "gender" : "Male",
  "age" : 50,
  "lectures" : [
    "C Language", "Advanced Programming Language",
    "HTML5 Programming", "Understanding to Game Engine",
    "Game Programming", "Mixed Reality"
  ]
}
```

정확성 검증을 위해 Google에서 [json 파일 포맷 검사] 검색 후 검증기 활용 가능.

Gemini Notebook

Processing JSON in Unity

Processing JSON

Unity 내장 JsonUtility 사용
(컴팩트한 최소 기능 제공)

외부 라이브러리 사용
(LitJSON, Newtonsoft.json
- 모든 JSON 기능 사용 시 권장)

JsonUtility 기초 설계 이론 - 분리 구조

👉 - 저장하려는 데이터 포맷 생성 -> PlayerData 객체로 분리하여 저장.

👉 - Data Container(데이터 컨테이너)와 Controller(실행 제어)의 철저한 분리 원칙 적용.

Gemini Notebook

Data Container vs Controller

```
1   public class DataManagerWithJsonUtility : MonoBehaviour {
2       // 유니티와 JSON이 주고받을 실질적 데이터
3       public PlayerData playerData;
4
5       void Start() {} void Update() {}
6   }
7
8   // 인스펙터 노출 안됨, 단순 정보 저장용
9   [System.Serializable]
10  public class PlayerData {
11      public string name;
12      public string affiliation;
13      public string gender;
14      public int age;
15      public string[] lectures;
16  }
```

분리 구조 장점

1. **코드 간결성**: 데이터를 가진 Container와 기능을 실행하는 Controller를 분리.
2. **불필요한 정보 차단**: Controller가 상속받는 MonoBehaviour 정보를 직렬화 과정에서 배제하여 순수 데이터만 파일로 저장 가능.

Gemini Notebook

JsonUtility: Serialize & Save

ToJson() 스크립트

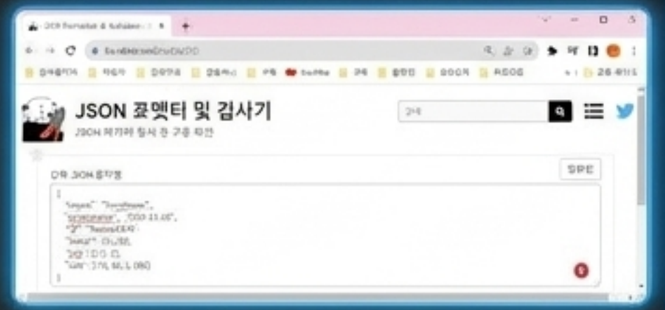
```
1   using System.IO;  // 파일 처리 기능 추가
2   public class DataManagerWithJsonUtility : MonoBehaviour {
3       public PlayerData playerData;
4
5       // 컴포넌트 메뉴에 실행 버튼 추가
6       [ContextMenu("To Json Data")]
7       public void WriteToJson() {
8           // 오브젝트를 포맷팅된 Json 문자열로 변환
9           string jsonData = JsonUtility.ToJson(playerData, true);
10
11          // 유니티 프로젝트 실행 경로(Application.dataPath)에 저장
12          string writeFileName = Path.Combine(Application.dataPath, "playerData.json");
13          File.WriteAllText(writeFileName, jsonData);
14      }
16  }
```

Gemini Notebook

JsonUtility: Load & Deserialize

FromJson() 스크립트

```
1   using System.IO;
2   public class DataManagerWithJsonUtility : MonoBehaviour {
3       public PlayerData playerData;
4
5       [ContextMenu("From Json Data")]
6       public void LoadPlayerDataFromJson() {
7           string readFileName = Path.Combine(Application.dataPath, "playerData.json");
8
9           // 파일 경로에서 문자열 데이터 읽어오기
10          string jsonData = File.ReadAllText(readFileName);
11
12          // JSON 문자열을 PlayerData 타입의 오브젝트로 역직렬화(제네릭 명시 필수)
13          playerData = JsonUtility.FromJson<PlayerData>(jsonData);
14      }
16  }
```



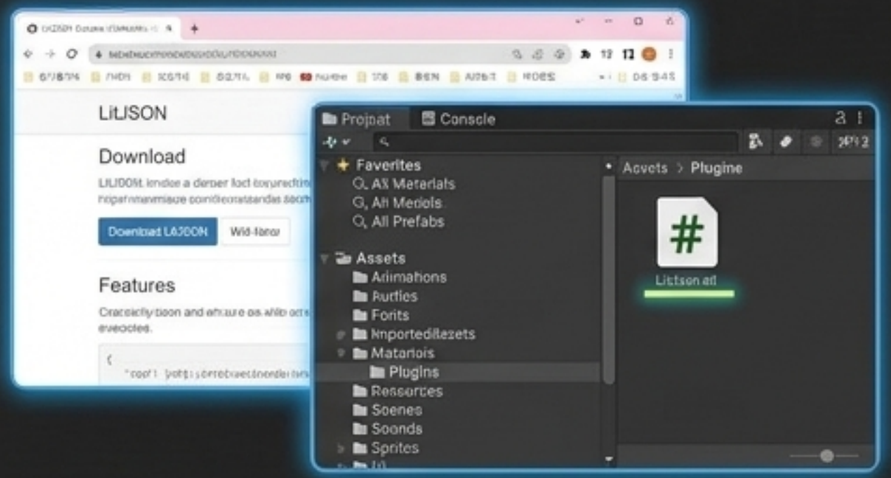
Gemini Notebook

Introducing External JSON Libraries

유니티 기본 JsonUtility는 기능이 제한적이므로 복잡한 데이터 구조 처리를 위해
외부 라이브러리 사용 권장.

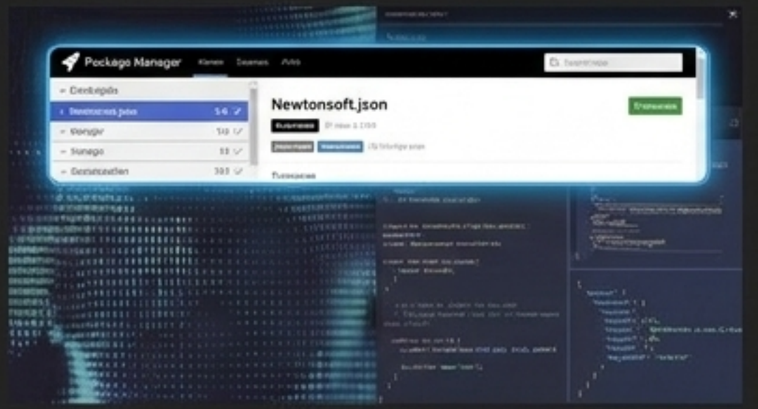
LitJSON

- 인터넷 검색 후 .dll 파일 다운로드.
- Asset/Plugins/LitJson.dll 경로로 수동 Import 필요.



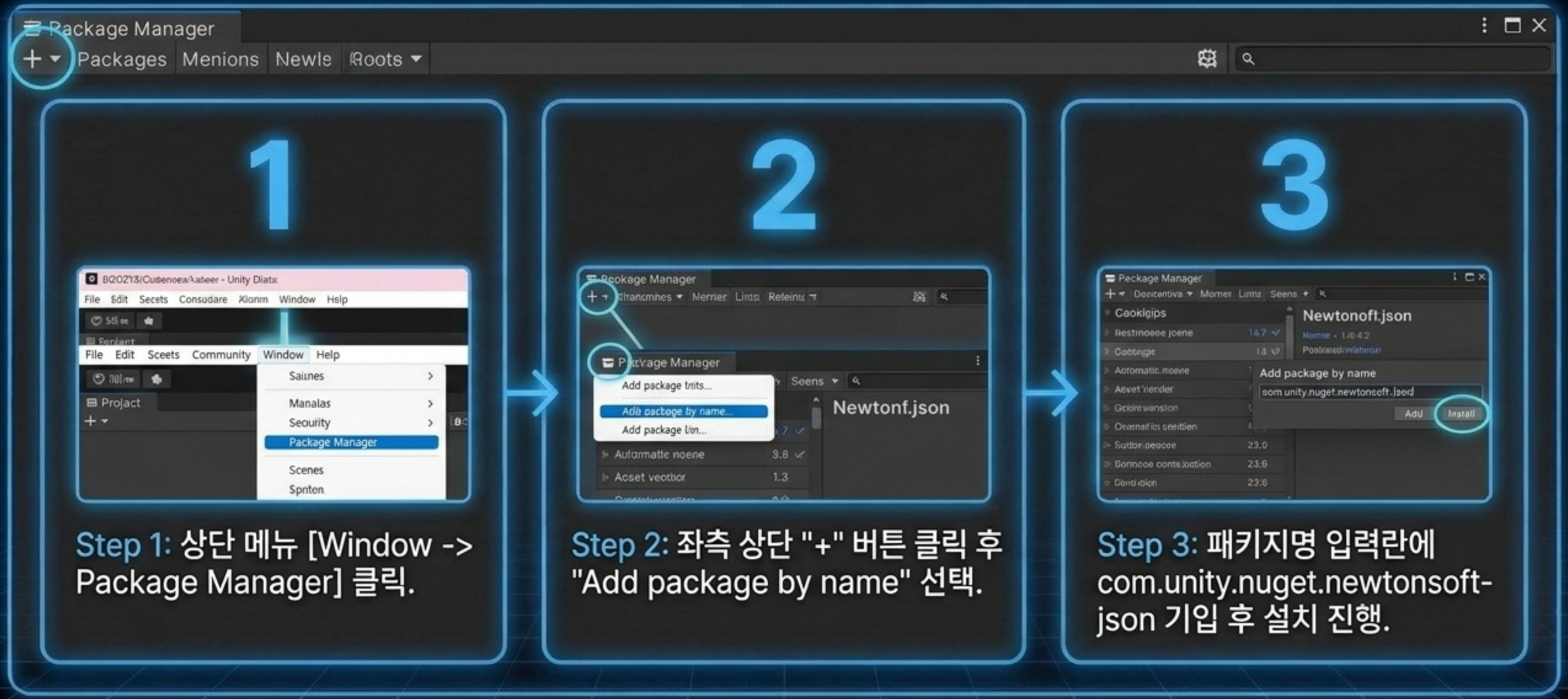
Newtonsoft.json

- Unity Package Manager를 통한 간편한 설치 (현업 표준 방식).
- 강력한 직렬화/역직렬화 및 복합 데이터(2중 배열 등) 완벽 지원.



Gemini Notebook

Installing Newtonsoft.json



Step 1: 상단 메뉴 [Window -> Package Manager] 클릭.

Step 2: 좌측 상단 "+" 버튼 클릭 후 "Add package by name" 선택.

Step 3: 패키지명 입력란에 com.unity.nuget.newtonsoft-json 기입 후 설치 진행.

Gemini Notebook

Newtonsoft: Basic Syntax

Serialize: Object to JSON

```
1
2   JsonConvert.SerializeObject(데이터)
3
4   클래스 객체를 완벽한 JSON 문자열로 직렬화.
```

Deserialize: JSON to Object

```
1
2   JsonConvert.DeserializeObject<클래스형식>(데이터)
3
4   JSON 문자열을 제네릭으로 선언된 지정 클래스 형식으로 역직렬화.
```

Gemini Notebook

Newtonsoft Basic Implementation

```
using Newtonsoft.Json;
using System.IO;

public class People {
    public string name; public int age;
    public People(string name, int age) { this.name = name; this.age = age; }
}

public class TestDataManager : MonoBehaviour {
    People p1 = new People("이옹환", 50);

    void Start() {
        // 직렬화 및 저장
        string jData = JsonConvert.SerializeObject(p1);
        string path = Path.Combine(Application.dataPath, "Object2Json.json");
        File.WriteAllText(path, jData);
        // 로드 및 역직렬화
        string readJsonData = File.ReadAllText(path);
        People p2 = JsonConvert.DeserializeObject<People>(readJsonData);
    }
}
```

Gemini Notebook

Advanced Serialization: Data Container

JsonDataClass 선언부

```
1  using Newtonsoft.Json;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  [System.Serializable]
6  public class JsonDataClass {
7      public int intNum; public float floatNum; public double doubleNum;
8      public string str; public bool boolVar;
9
10     public int[] intArray;
11     public string[] strArray;
12     public List<int> intList = new List<int>();
13     public Dictionary<string, int> dicGameScore =
14         new Dictionary<string, int>();
15     public Dictionary<string, string> dic =
16         new Dictionary<string, string>();
17     public Vector3Int vecInt;
18 }
```

Gemini Notebook

Advanced Serialization: Constructor Allocation

생성자 구현부

```
public JsonDataClass() {
    intNum = 100; floatNum = 3.4f; doubleNum = 123.456;
    str = "JSON Test String"; boolVar = true;
    intArray = new int[] { 1, 2, 3, 4, 5, 6, 7 };
    strArray = new string[] { "LG", "KT", "NC", "SSG", "두산" };
    for (int i=0; i<5; i++) { intList.Add(i*10); }
    dicGameScore.Add("LG", 1); dicGameScore.Add("KT", 2); dicGameScore.Add("NC", 3);
    dic.Add("WK", "Wonkwang University");
    dic.Add("WKDC", "Wonkwang Digital Content");
    dic.Add("WKGC", "Wonkwang Game Content");
    vecInt = new Vector3Int(1, 2, 3);
}
```

Gemini Notebook

Advanced Serialization: Data Output Validation

PrintData 출력부

```
    public void PrintData() {
        Debug.Log($"integer = {intNum}"); Debug.Log($"float = {floatNum}");
        Debug.Log($"double = {doubleNum}"); Debug.Log($"str = {str}"); Debug.Log($"boolVar = {boolVar}");
        for (int i=0; i<intArray.Length; i++) { Debug.Log($"int Array[{i}] = {intArray[i]}"); }
        for (int i = 0; i < strArray.Length; i++) { Debug.Log($"string Array[{i}] = {strArray[i]}"); }
        foreach(var item in intList) { Debug.Log($"int List: {item}"); }
        foreach(var item in dicGameScore) {
                                        Debug.Log($"int Dictionary: key={item.Key} / value={item.Value}");}
        foreach (var item in dic) {Debug.Log($"string Dictionary: key={item.Key} / value={item.Value}");}
        Debug.Log($"int Vector = {vecInt}");
    }
}
```

Gemini Notebook

Advanced Serialization: File IO Execution

DataManagerWithNewtonSoftJson 구현부

```
public class DataManagerWithNewtonSoftJson : MonoBehaviour {
    public JsonDataClass jsonData;
    void Start() { jsonData = new JsonDataClass(); }

    public void WriteToJsonFile() {
        string savingObjectToJson = JsonConvert.SerializeObject(jsonData);
        string path = Path.Combine(Application.dataPath, "ObjectToJson.json");
        File.WriteAllText(path, savingObjectToJson);
    }

    public void ReadFromJsonFile() {
        string path = Path.Combine(Application.dataPath, "ObjectToJson.json");
        string readJsonData = File.ReadAllText(path);
        JsonDataClass createObjectFromJson = JsonConvert.DeserializeObject<JsonDataClass>(readJsonData);
        createObjectFromJson.PrintData();
    }
}
```

Gemini Notebook

Framework Synthesis: Built-in vs Newtonsoft

Diagnostic Comparison Table UI

[Feature]	Unity JSON Utility	Newtonsoft.Json
[네임스페이스]	UnityEngine (추가 불필요)	Newtonsoft.Json (패키지 설치 및 추가 필요)
[직렬화 (Object->Json)]	JsonUtility.ToJson(데이터)	JsonConvert.SerializeObject(데이터)
[역직렬화 (Json->Object)]	JsonUtility.FromJson<형식>(데이터)	JsonConvert.DeserializeObject<형식>(데이터)
[다차원 배열 지원]	지원 안함	완벽 지원

```
{
  "scores" : [ [0,1], [2,3,4] ]
}
```

이중 배열 지원 예시 – TempJson.json: Newtonsoft.Json은 위와 같은 다차원/가변 배열도 문제없이 직렬화 가능.

Gemini Notebook